

PENETRATION TESTING REPORT

OWASP Juice Shop - Web Application Security Test

Made by: Seif eldeen mahmoud

Graduation Project

Date: 26/8/2026

Machine: OWASP Juice Shop

1. Executive Summary

This report shows the security test on the OWASP Juice Shop web application. The goal was to find security problems (vulnerabilities) and check how bad they are. We found 6 problems. Two of them are very serious (Critical). The tests happened on a local copy of the application, not a real company website.

Findings Overview

#	Finding	OWASP Category	Severity
1	SQL Injection - Login Bypass	A03:2021 - Injection	Critical
2	SQL Injection - Data Extraction	A03:2021 - Injection	Critical
3	Sensitive Data Exposure - whoami API	A01:2021 - Broken Access Control	High
4	Cross-Site Scripting (XSS)	A03:2021 - Injection	Medium
5	Broken Access Control - Open FTP Folder	A01:2021 - Broken Access Control	Medium
6	Sensitive Data Exposure - package.json File	A05:2021 - Security Misconfiguration	Low

2. Scope and Method

Scope

Target: OWASP Juice Shop (local copy - <http://localhost:3000>)

Test type: Black-box and gray-box (with and without a user account)

Method

We followed the OWASP Top 10 (2021) list. This list shows the most common security problems in web applications. Each finding in this report has an OWASP category. We used these steps:

- Look at the website and find all pages and forms
- Test the login page for weak security
- Test input fields (like search) for bad code
- Check if normal users can see admin pages or files
- Write down every problem with proof (screenshots)

Tools Used

- Firefox Browser and Developer Tools (Network tab)
- Manual testing (typing payloads by hand)

3. Detailed Findings

Each finding below shows: what the problem is, how bad it is (severity), the steps we did, proof (screenshots in order), and how to fix it.

Finding 1: SQL Injection - Login Bypass

OWASP Category	A03:2021 - Injection	Severity: Critical
CVSS 3.1 Score	9.8 (Critical)	

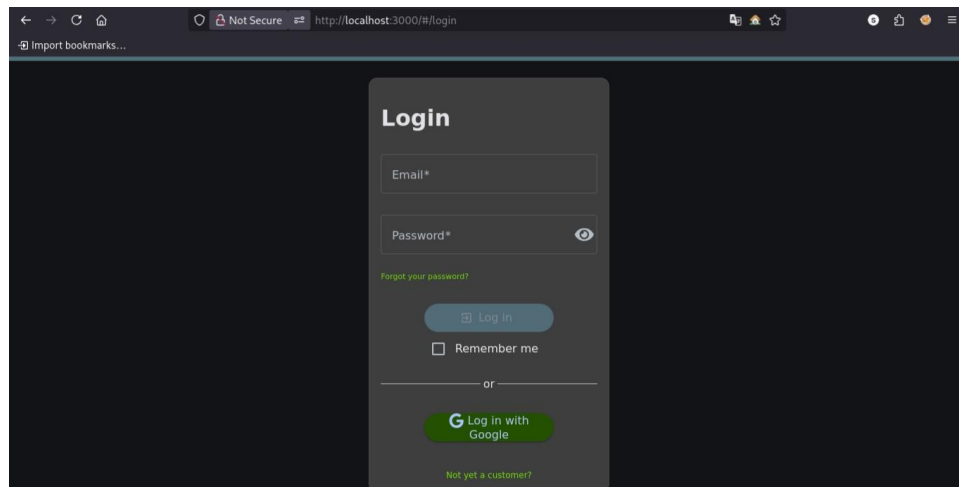
What is the problem?

The login page does not check the text a user types in a safe way. We typed special SQL code in the email box. The website ran our code by mistake. This let us log in as the Admin user, without knowing the real password.

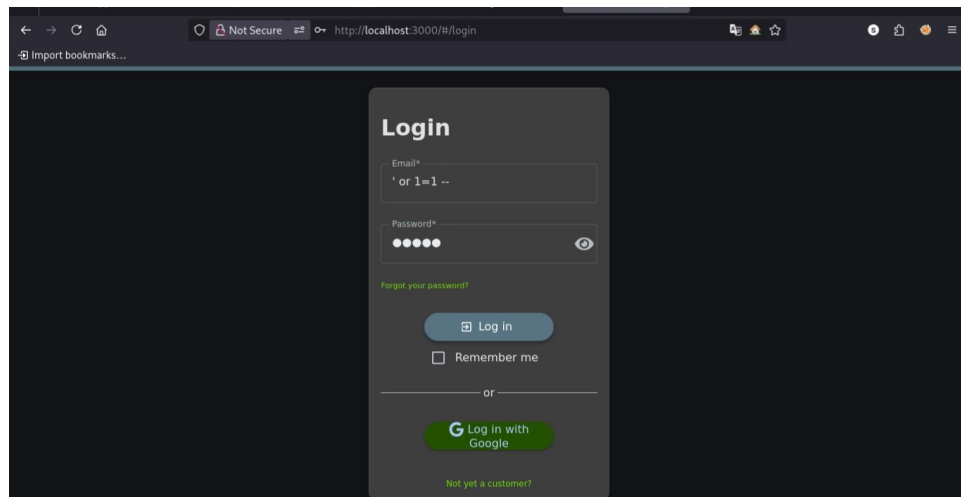
Steps we did

- Step 1: We opened the login page.
- Step 2: In the email box, we typed: ' or 1=1 --
- Step 3: We typed anything in the password box and clicked Log in.
- Step 4: The website logged us in as the Admin user.
- Step 5: We opened the Admin page and saw private data (all user emails).

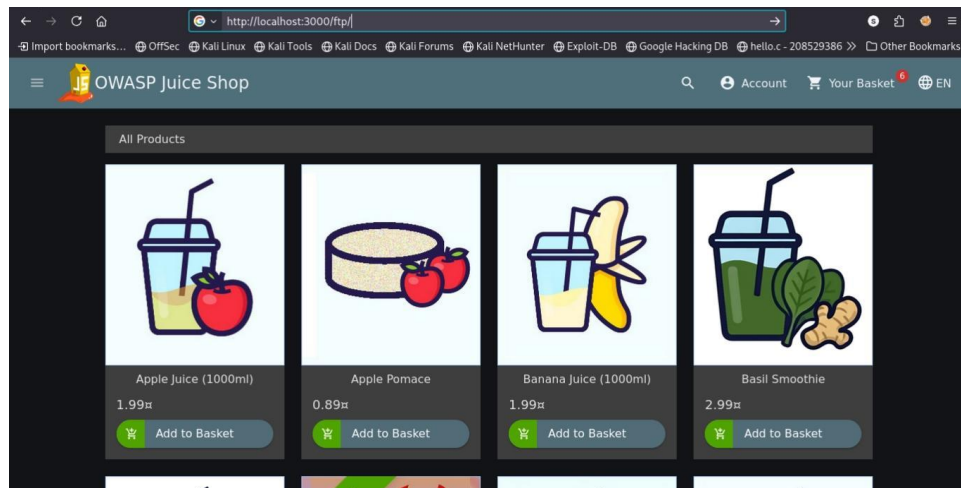
Proof (Screenshots, in order)



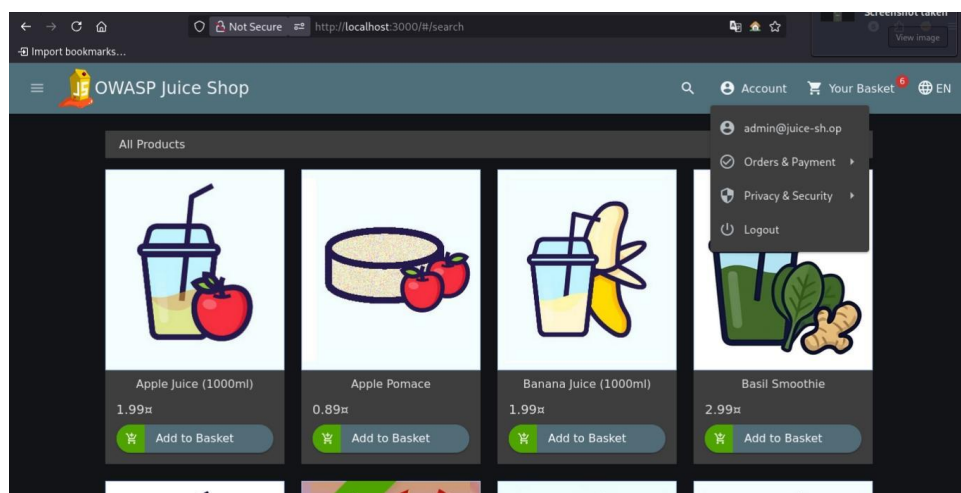
Step 1 - The empty login page, before the attack.



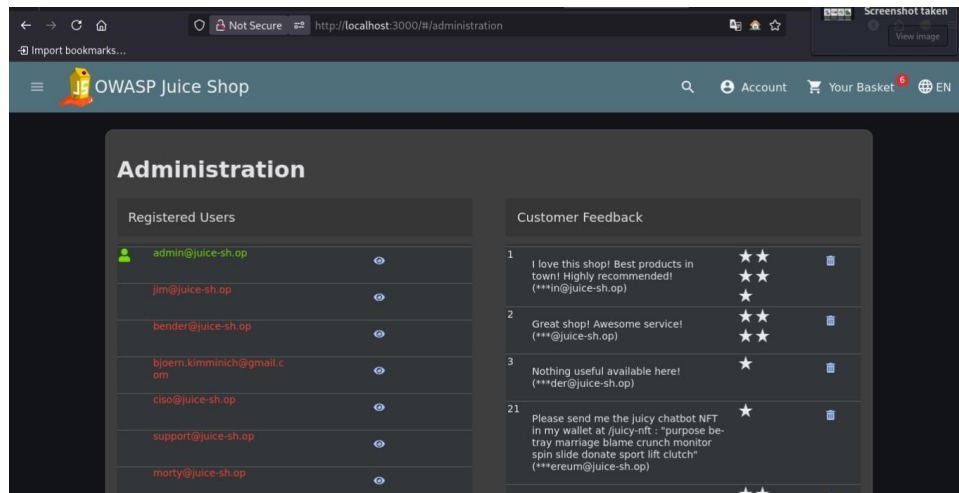
Step 2 - We typed the SQL Injection code in the email box: ' or 1=1 --



Step 3 - The login worked. We are now inside the website as a logged-in user.



Step 4 - The account menu shows we are logged in as admin@juice-sh.op.



Step 5 - We can open the Admin page and see a list of all users.

Impact (why this is dangerous)

An attacker does not need a password. The attacker can log in as Admin and control the whole website. The attacker can see, change, or delete other people's data.

Recommendation (how to fix it)

- Use parameterized queries (prepared statements). Never build SQL text by joining strings from user input.
- Check and clean all user input before using it.
- Use an ORM library (like Sequelize) the safe way.

Finding 2: SQL Injection - Data Extraction (UNION-based)

OWASP Category	A03:2021 - Injection	Severity: Critical
CVSS 3.1 Score	9.1 (Critical)	

What is the problem?

We used a stronger SQL Injection payload with the word UNION SELECT. This payload asks the database to add extra data (like emails and password hashes) into the normal search results. The website showed this secret data to us.

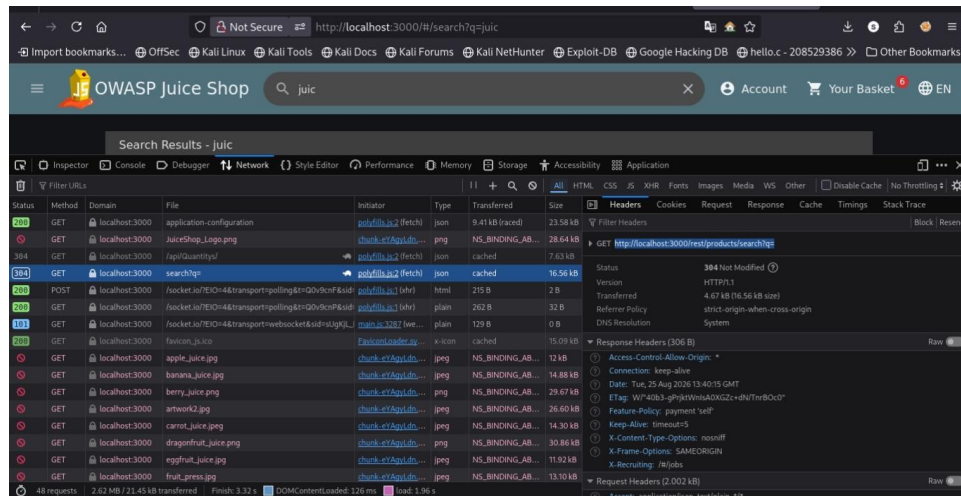
Steps we did

- Step 1: We opened the Network tab in Developer Tools, to see the requests.
- Step 2: We sent a request with a UNION SELECT payload in the search box.
- Step 3: We looked at the response data from the server.
- Step 4: The response showed user emails and password hashes.

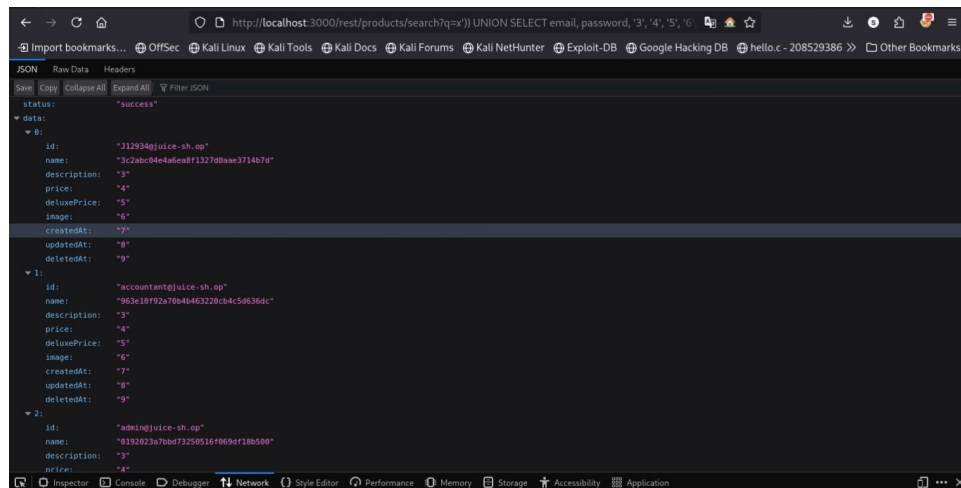
Payload Used

```
'x')) UNION SELECT email, password, '3', '4', '5', '6', '7', '8', '9' FROM Users--
```

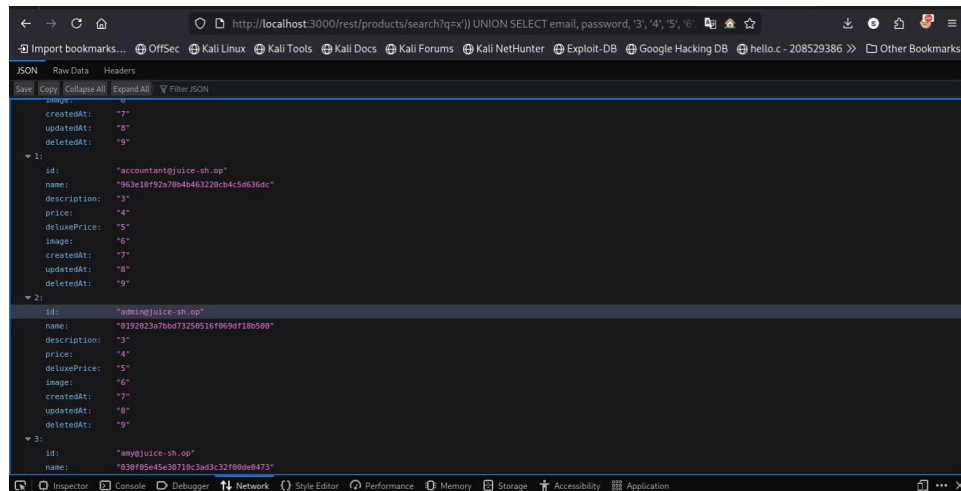
Proof (Screenshots, in order)



Step 1 - The Network tab, watching the requests to the server.



Step 2-3 - We sent the UNION SELECT payload. The response comes back with extra data.



Step 4 - The response shows real user emails and password hashes. This is secret data.

Impact (why this is dangerous)

An attacker can read the whole database. This means all user emails and password hashes. If a password is weak, the attacker can crack the hash and get the real password.

Recommendation (how to fix it)

- Same fix as Finding 1: use parameterized queries.
- Never send password hashes to the client, even by mistake.
- Use a Web Application Firewall (WAF) as an extra layer of defense.

Finding 3: Sensitive Data Exposure via API Parameter Manipulation (whoami)

OWASP Category	A01:2021 - Broken Access Control	Severity: High
CVSS 3.1 Score	7.5 (High)	

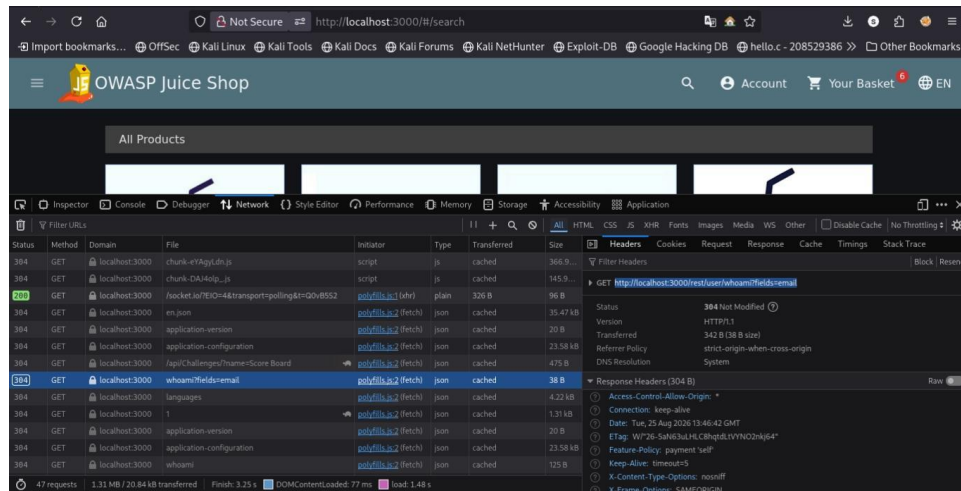
What is the problem?

The website has an API address called whoami. It normally shows the email of the logged-in user, like this: `/rest/user/whoami?fields=email`. We changed the word email to password in the address. The server did not check if this is allowed. It sent back the password hash of the user. This is not SQL Injection. This is a different problem: the server trusts the client too much, and does not control which fields a user can ask for.

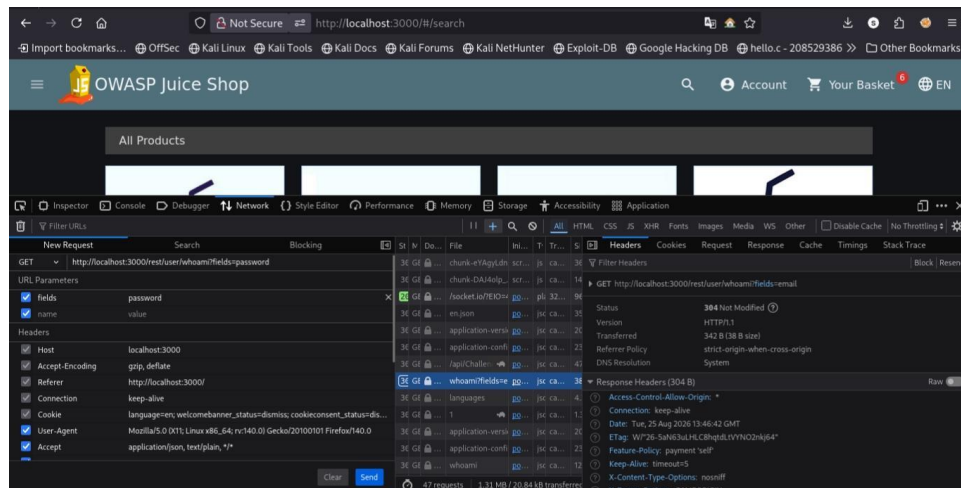
Steps we did

- Step 1: We saw a normal request to `/rest/user/whoami?fields=email` in the Network tab.
- Step 2: We opened this request and changed the value of the fields parameter, from email to password.
- Step 3: We sent the new request. The server answered with status 200 OK.
- Step 4: The response body showed the password hash of the user.

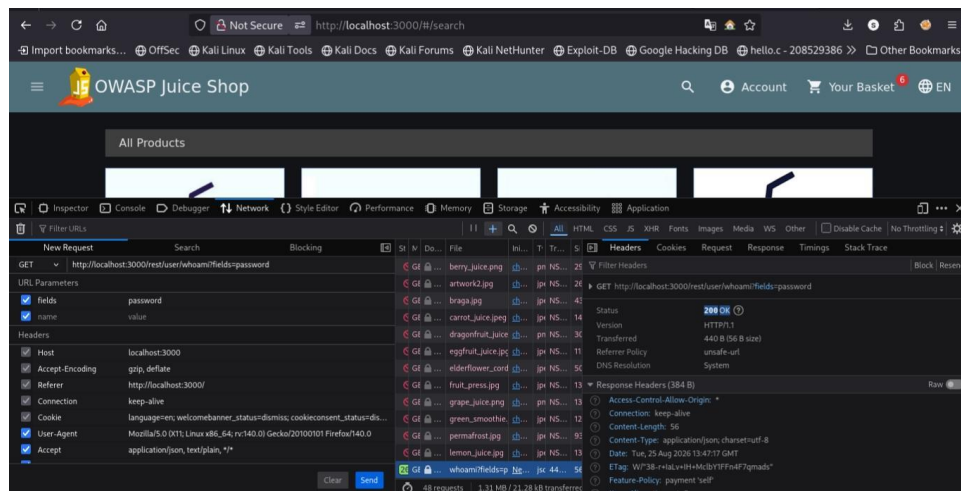
Proof (Screenshots, in order)



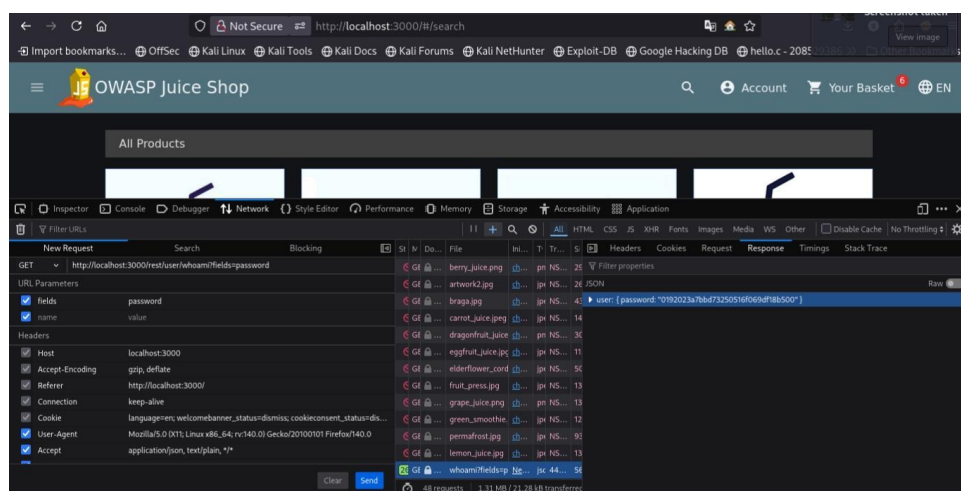
Step 1 - The normal request: `/rest/user/whoami?fields=email`



Step 2 - We change the fields parameter value from email to password.



Step 3 - We send the new request. The server answers 200 OK.



Step 4 - The response shows the password hash: 0192023a7bbd73250516f069df18b500

Impact (why this is dangerous)

An attacker can ask the server for any field of the user object, not only email. This can leak password hashes and other private data, without any SQL Injection. The attacker only needs to change one word in the URL.

Recommendation (how to fix it)

- On the server, make a fixed (allow-listed) set of fields that this API can return, like only email and name.
- Never allow the client to choose which database fields it can see.
- Never send password hashes in any API response, even by mistake.

Finding 4: Cross-Site Scripting (Dom-XSS)

OWASP Category	A03:2021 - Injection	Severity: Medium
CVSS 3.1 Score	6.1 (Medium) -	

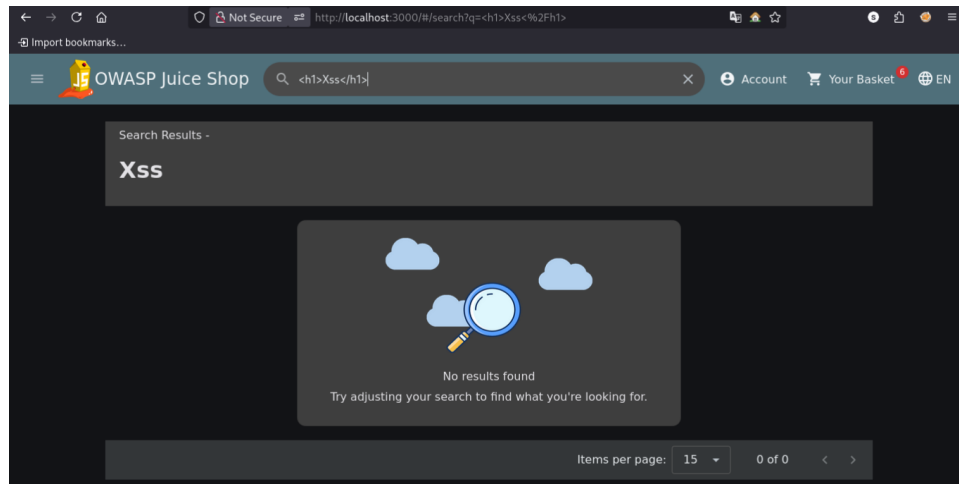
What is the problem?

The search box does not clean the text a user types. We typed a small piece of code (a script) in the search box. The website ran our code in the browser. This is called Cross-Site Scripting, or XSS.

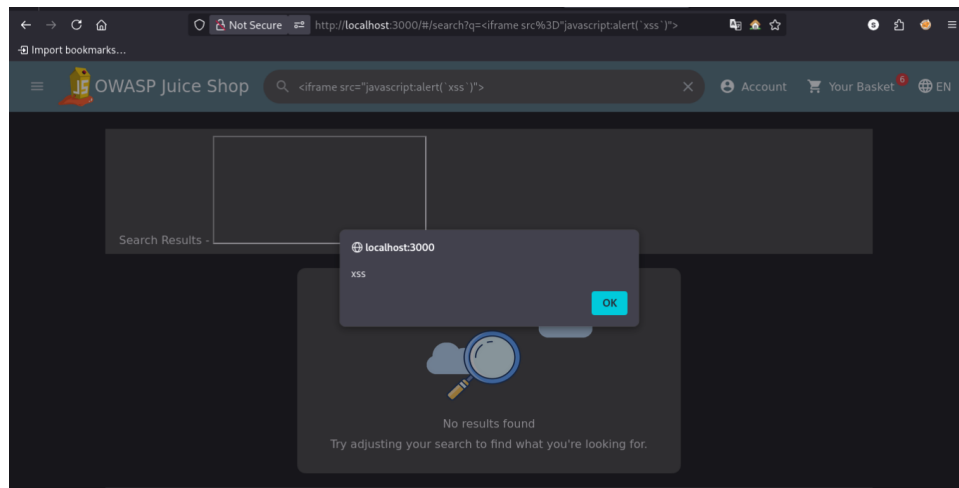
Steps we did

- Step 1: We typed a script payload in the search box.
- Step 2: The website showed a pop-up alert box. This proves the code ran.
- Step 3: We tried a stronger payload, with an iframe tag.
- Step 4: The website showed a video inside the search results page. This is not normal.

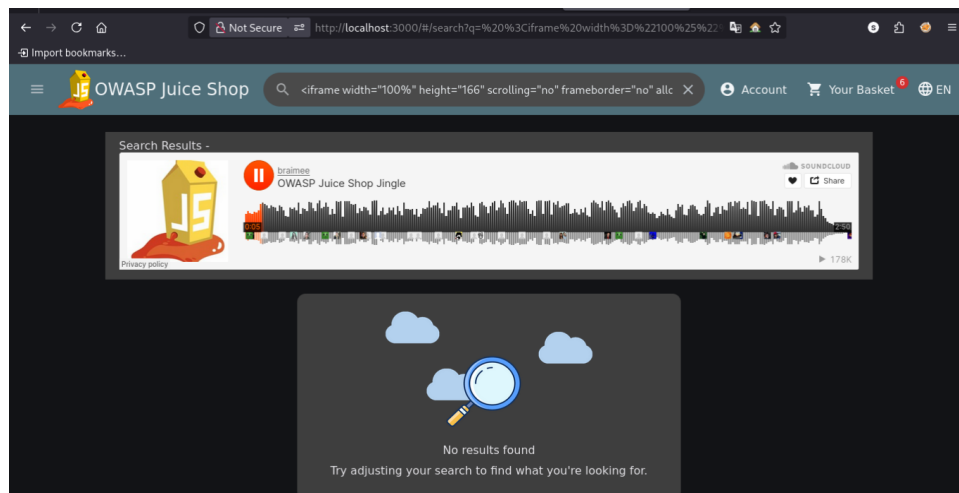
Proof (Screenshots, in order)



Step 1 - We typed a script payload in the search box.



Step 2 - A pop-up alert box appears. This means our code ran (XSS works).



Step 3-4 - A stronger payload adds a video (iframe) inside the page.

Impact (why this is dangerous)

An attacker can send a link with this payload to a victim. When the victim opens the link, the attacker's code runs in the victim's browser. The attacker can steal login session data or show fake content.

Recommendation (how to fix it)

- Clean (sanitize) all user input before showing it on the page.
- Encode output text, so code cannot run (output encoding).
- Use a Content Security Policy (CSP) header to block bad scripts.

Finding 5: Broken Access Control - Open Folder

OWASP Category	A01:2021 - Broken Access Control	Severity: Medium
CVSS 3.1 Score	5.3 (Medium)	

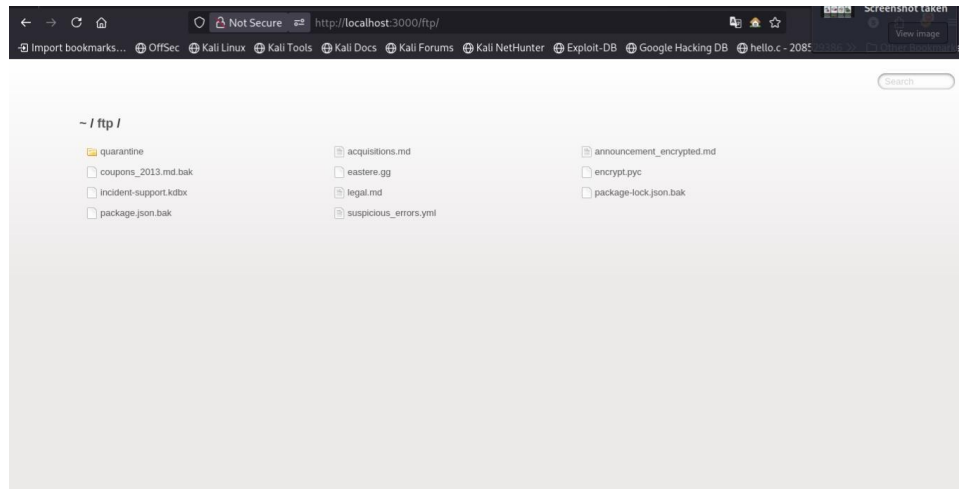
What is the problem?

We found a folder on the website at the address `/ftp/`. This folder should be private, only for the company. But anyone can open it and see the file list, with no login needed.

Steps we did

- Step 1: We opened the address `http://localhost:3000/ftp/` in the browser.
- Step 2: The website showed a full list of files, with no login check.

Proof



The /ftp/ folder is open. Anyone can see the files inside.

Impact (why this is dangerous)

An attacker can see and download private company files. Some files may have secret information.

Recommendation (how to fix it)

- Turn off directory listing on the web server.
- Add a login check (authentication) before this folder.
- Move private files outside of the public web folder.

Finding 6: Sensitive Data Exposure - package.json File

OWASP Category	A05:2021 - Security Misconfiguration	Severity: Low
CVSS 3.1 Score	4.3 (Medium-Low)	

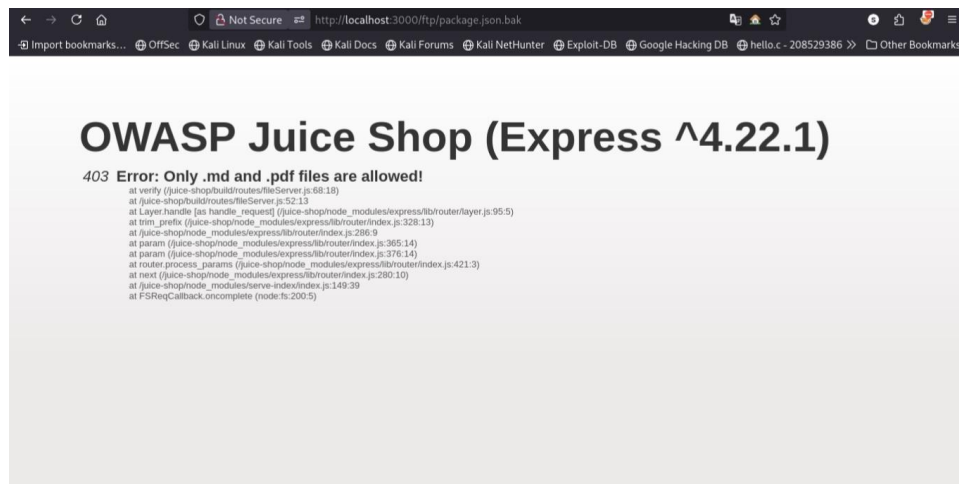
What is the problem?

The file upload page only allows .md and .pdf files. We tried a different file type. The server showed an error message with technical details (a stack trace). This error message helped us find and download a file called package.json, which lists the software the website uses.

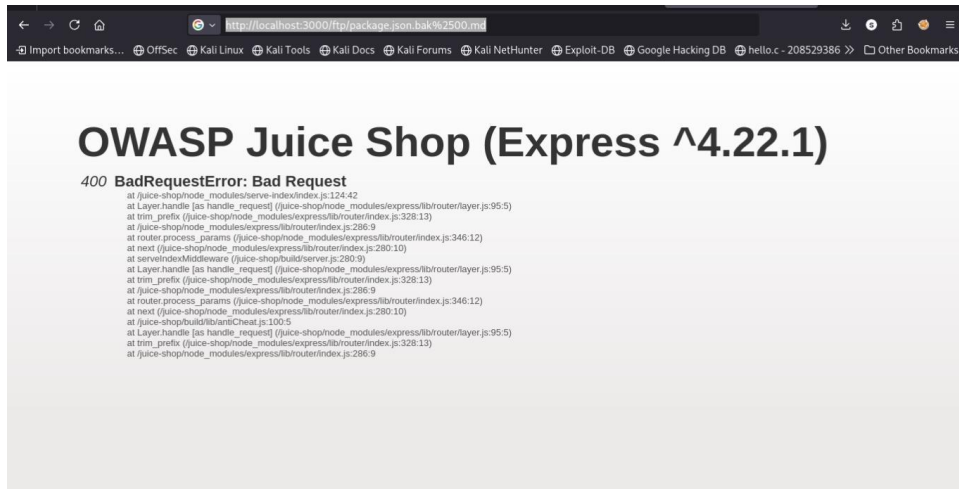
Steps we did

- Step 1: We tried to upload a file type that is not allowed.
- Step 2: The server showed a detailed error message (stack trace).
- Step 3: Using this information, we downloaded the file package.json.
- Step 4: We opened the file and saw the software name, version, and other technical data.

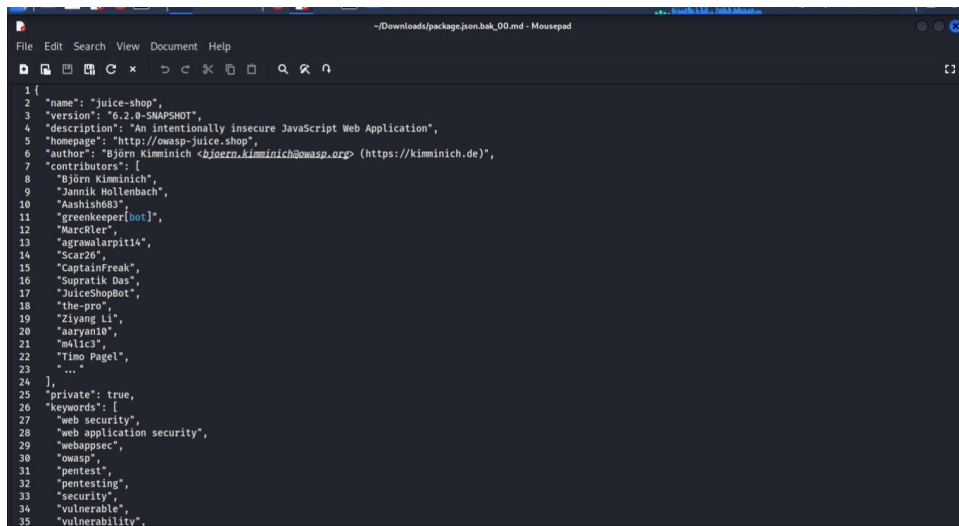
Proof (Screenshots, in order)



Step 1-2 - Error message: only .md and .pdf files are allowed.



Step 2 - A second error message with technical stack trace details.



Step 3-4 - The downloaded package.json file. It shows software name and details.

Impact (why this is dangerous)

The error messages and the package.json file give an attacker useful information about the software and its version. The attacker can search for known security problems (CVEs) for this exact version, and use them to attack the website.

Recommendation (how to fix it)

- Show simple, generic error messages to users. Do not show stack traces.
- Block public access to configuration files like package.json.
- Keep all software packages updated to the newest safe version.

4. Conclusion

The Juice Shop application has several serious security problems. The most dangerous ones are the two SQL Injection findings, because an attacker can log in as Admin and read the full database without a password. The team should fix the Critical and High findings first. After that, fix the Medium and Low findings. Regular security testing will help keep the application safe in the future.

Appendix A: OWASP Top 10 (2021) Reference

- A01:2021 - Broken Access Control
- A02:2021 - Cryptographic Failures
- A03:2021 - Injection
- A04:2021 - Insecure Design
- A05:2021 - Security Misconfiguration
- A06:2021 - Vulnerable and Outdated Components
- A07:2021 - Identification and Authentication Failures
- A08:2021 - Software and Data Integrity Failures
- A09:2021 - Security Logging and Monitoring Failures
- A10:2021 - Server-Side Request Forgery (SSRF)